Real-time algorithm development for underwater bot detection and localization

A
Thesis Submitted
in Partial Fulfillment of the Requirements
for the Degree of
MASTER OF TECHNOLOGY
By

JITHU J
Under the Supervision of Dr. CHAYAN BHAWAL

Electronics and Electrical Department
Indian Institute of Technology Guwahati
May, 2025

Declaration

This is to certify that the thesis entitled " **Real-time algorithm development for underwater bot detection and localization**", submitted by me to the *Indian Institute of Technology Guwahati*, for the award of the degree of Master of Technology, is a bonafide work carried out by me under the supervision of Dr. Chayan Bhawal. The content of this thesis, in full or in parts, have not been submitted to any other University or Institute for the award of any degree or diploma. I also wish to state that to the best of my knowledge and understanding nothing in this report amounts to plagiarism.

Signed: _____

Jithu J
Electronics and Electrical Department,
Indian Institute of Technology Guwahati,
Guwahati-781039, Assam, India.

Date: _____

Certificate

This is to certify that the thesis entitled " **Real-time algorithm development for underwater bot detection and localization**", submitted by Jithu J (234102314), a master's student in the *Electronics and Electrical Department*, *Indian Institute of Technology Guwahati*, for the award of the degree of Master of Technology, is a record of an original research work carried out by him under my supervision and guidance. The thesis has fulfilled all requirements as per the regulations of the institute and in my opinion has reached the standard needed for submission. The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Signed: _____

Supervisor: Dr. Chayan Bhawal
Electronics and Electrical Department,
Indian Institute of Technology Guwahati,
Guwahati-781039, Assam, India.

Date: _____

Acknowledgements

I would like to extend my sincere gratitude to Dr. Chayan Bhawal, my thesis supervisor, for his exceptional guidance and support throughout my research journey. His insightful feedback and steadfast patience have been instrumental in helping me navigate various challenges and greatly enhancing the quality of my work.

I would like to express my sincere gratitude to Dr. Anirban DasGupta for his steadfast support throughout my thesis journey. His technical expertise, insightful perspectives, and readiness to assist me during challenging moments have been invaluable.

I would like to express my heartfelt gratitude to the faculty and staff of the Electronics and Electrical Engineering Department for fostering a supportive academic environment and providing access to essential resources. I also acknowledge Parvez, Vishal, Prasad, and Divya for their invaluable suggestions and insightful discussions.

I wish to extend my sincere gratitude to my friends and colleagues, whose unwavering moral support and occasional distractions enriched my research journey, making it both enjoyable and less burdensome.

Furthermore, I am profoundly appreciative of my family for their boundless love, understanding, and encouragement. Their steadfast belief in my potential has served as my most significant source of motivation.

This thesis would not have been achievable without the invaluable support and contributions of these individuals, to whom I am earnestly thankful.

Sincerely
Jithu J

Abstract

This thesis presents the design of a real-time object detection and localization system for underwater robotic applications optimized for execution on low-resource systems such as edge platforms. The proposed system is a video pipeline that integrates an Intel RealSense depth camera, an inertial measurement unit (IMU), and an optimized version of YOLOv11 for accurate detection and localization of underwater targets in 3D space.

The framework incorporates ByteTrack for robust object tracking and utilizes temporal depth smoothing through a moving average filter to improve depth stability. A Mahony filter-based IMU fusion is employed to efficiently project the object position into the world frame to compute the source orientation in real-time. This transformation maps the 3D point cloud into the world coordinate frame.

The proposed system integrates advanced postprocessing techniques for enhancing depth and color image streams. To mitigate noise typical of underwater environments, the system employs a range of image processing methodologies, including Gaussian blur, Contrast-Limited Adaptive Histogram Equalization (CLAHE), and bilateral filters. These techniques collectively enhance visibility and improve object detection capabilities. Furthermore, the processing pipeline is optimized for efficient performance on GPUs with restricted memory access, making it particularly well-suited for deployment on edge AI platforms utilized in autonomous underwater vehicles (AUVs). This design consideration ensures that the system can operate effectively in real-time scenarios while maintaining high-quality image output.

This research presents a scalable and modular pipeline designed for underwater perception, facilitating real-time situational awareness and precise object-level localization. These advancements are essential for a variety of tasks, including autonomous inspection, navigation, and object manipulation within subaquatic environments. The proposed methodology enhances the ability to operate effectively in challenging underwater conditions, thereby broadening the scope of potential applications in marine exploration and underwater robotics.

Contents

List of Figures

List of Tables

Chapter 1
Introduction

OCEANS are still one of the biggest mysteries of the planet. Many of the ocean beds are still unmapped and inaccessible to humans. The extreme conditions make it difficult for physical exploration. Due to a lack of surveillance, the underwater world remains a curiosity and, at the same time, a defense threat. Criminals can use it to smuggle illegal items into the country. Technological advancement has helped us access such places with the help of robots. However, due to a lack of effective methods for navigation, object detection, etc, the problems persist.

The introduction of deep neural networks in object detection has made groundbreaking changes in underwater exploration. These networks make it possible to process large amounts of data with limited resources. Deep neural networks can be used to find a solution to this problem.

1.1 Problem Statement

The recent tragedy involving the Titan submersible, operated by OceanGate, which imploded during its expedition to the Titanic wreck site, highlights critical safety and operational vulnerabilities in underwater exploration. This incident not only resulted in

the loss of lives, but also underscores the urgent need for the integration of autonomous robotic systems into marine environments for surveying, rescue operations, and other essential functions.

Current underwater autonomous bots face significant challenges primarily related to limitations in power and computing resources, which constrain their operational effectiveness and functionality in deep-sea environments. These limitations impede the ability of autonomous systems to perform complex tasks, navigate effectively, and ensure the safety of operations in unpredictable conditions.

As such, there is a pressing need for the development of lightweight algorithms that enhance the navigation and surveillance capabilities of these bots. By addressing the challenges of power efficiency and computational resource optimization, these algorithms can contribute to the advancement of autonomous underwater systems, ultimately improving their reliability and effectiveness in performing critical missions. This problem statement emphasizes the need for innovative solutions that leverage autonomous technology to mitigate risks and improve the safety and efficacy of underwater exploration and intervention.

1.2 Aims and Objectives

1. To develop a robust dataset containing images of objects captured in underwater environments.

2. To train and optimize a suitable object detection algorithm to detect underwater objects.

3. To integrate sensors to enable position tracking and orientation awareness of the system.

4. To develop a real-time video processing pipeline that integrates the object detection model, camera module, and sensors for underwater object localization.

5. To enhance detection accuracy and processing speed by optimizing the object detection algorithm.

1.3 Solutions Approach

To address the challenges of real-time underwater object detection and localization, the following approach is adapted:

1. **Dataset Development**: A setup consisting of a depth camera enclosed in a submersible enclosure is developed. The setup is used to capture images of various scenes. The images are annotated and augmented to produce the dataset.

2. **Model selection and Optimization**: A suitable object detection model is selected and finetuned on the dataset. The selected model is optimized using quantization techniques to increase the inference speed.

3. **IMU integration**: sensor fusion is implemented using a suitable algorithm to estimate orientation and transform object coordinates from camera to world frame.

4. **Depth estimation**: The depth of the object is calculated from the region of interest given by the object detection model.

5. **Real-Time Processing pipeline**: All the modules are combine into a low latency video processing pipeline capable of running in resource constrained conditions.

Chapter 2
Literature Review

OBJECT detection is one of the most researched topics in the world. The classic object detection methods employed template matching which compared segments of image with a preset image template or its histogram to detect objects. One such approch is done by Viola and Jones (2001)[1] introduced the Haar-like features for face detection, which significantly reduced computation time. They used AdaBoost for feature selection and a cascade classifer to improve detection efficiency.

In recent years, there have been groundbreaking results in object detection models. Adopting deep neural networks, particularly convolutional neural networks (CNNs), in this area has significantly increased detection accuracy and speed. Region-based CNN (R-CNN)[2] uses selective search to predict regions of interest(ROIs) and feed them through CNN for classification. RCNN is a two-stage network. The RCNN models successfully reduced detection time but needed to be faster for real-time detection. Yolo (You Only Live Once) is a single-stage object detection model introduced in 2015 by Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi in their famous research paper "You Only Look Once: Unified, Real-Time Object Detection" [3]. Yolo considers object detection to be a single regression. It predicts bounding boxes in a single forward pass of the network, thus making it significantly faster than classic object detection models. Yolo has three parts: backbone (does feature extraction, CSPDarknet53 is used in YoloV5

), neck (aggregates feature maps and prepare it for final prediction, PANet is used in YoloV5) and head (responsible for final prediction, YoloV5 uses a custom head).

Yolo v11 by Ultralytics [4] is the newest version of Yolo available for general use. It is pre-trained on COCO dataset. The newest version achieves high Mean Average Precision with significantly fewer parameters. Utralytics develops and releases newer versions of YOLO model. Ultralytics library makes it easier to custom-train models with their trained checkpoints.

Even though object detection models are far advanced, they have significant processing and power requirements. Therefore, performing object detection in real-time on stand-alone systems with limited resources, like a bot, takes much work. The model can be scaled down, removing unnecessary hidden layers and weights and tailor-fitting the network to the application. Even then, the Mean Average Precision is significantly affected. We have technologies such as Nvidia TensorRT SDK, which helps optimize the models for high-performance inference.

The accuracy issue related to the model can be reduced further with the help of a well-developed dataset. Roboflow [5] is an open-source platform that helps with this. It has a user-friendly interface allows us to annotate, augment, and test the dataset seamlessly.

Real-time object detection is a research topic that has garnered much interest recently. This technology has numerous applications in marine preservation, underwater navigation, defense, and surveillance. Classic models, such as FRCNN[6], were first used for underwater object detection. The significant challenges in this area are environmental factors such as light scattering, distortion, restriction on computing resources. They achieved impressive mAp scores, but the model latency was high, making it unfit for deployment on edge devices.

Single-stage models offer better solutions for latency concerns. Yolo models are about 100 times faster than the classic models[7, 8]. Yolo model achieves up to 155 frames per second (fps), while FRCNN struggles to reach real-time speed even on large computing equipment. Multiple approaches using YOLO models for underwater object detection

are present. One such approach is given by Wang et al. (2022)[9]. They proposed a model named ULO or Yolo nano underwater, a modified version of yoloV3. Their model achieved an mAp of 53.48.

A newer approach using a YoloV5 model is proposed by Aditya et al. (2023)[10]. They introduced a deep wave net image enhancing algorithm to denoise the images before the images are passed to Yolo models. This improved the mAp up to 90.3 for a YoloV5 model. Yibing et al. (2024)[11] proposed a model based on the new YoloV8 model specifically for edge computing. They proposed an optimized YoloV8n model through pruning, quantization, and knowledge distillation. Achieveing an mAp of 58.1.

Chapter 3
Methodology

THIS chapter outlines the methodology followed in development of the pipeline.

3.1 Hardware Configuration

The device used for data acquisition and pipeline input is Intel real sense d435i depth camera. The device features several sensors and have a good community support along with a well maintained software development kit. It connects seamlesly with most devices. the list of sensors integrated into the device is:

1. Stereo depth sensors (Two monochrome global shutter cameras). resolution up to 1280×720 @ 30 fps.
2. Color camera (RGB Sensor). Resolution upto 1920×1080 @ 30 fps.
3. IMU Inertial Measurement unit (3-axis gyroscope and 3-axis accelerometer). Frequency upto 200Hz.

The camera is not inherently waterproof. It does not hold an official IP rating, which means it is neither dustproof nor waterproof. Since the data should be acquired underwater, the device is enclosed in a glass enclosure. This robust, transparent cuboidal

FIGURE 3.1: Intel realsense d435i camera

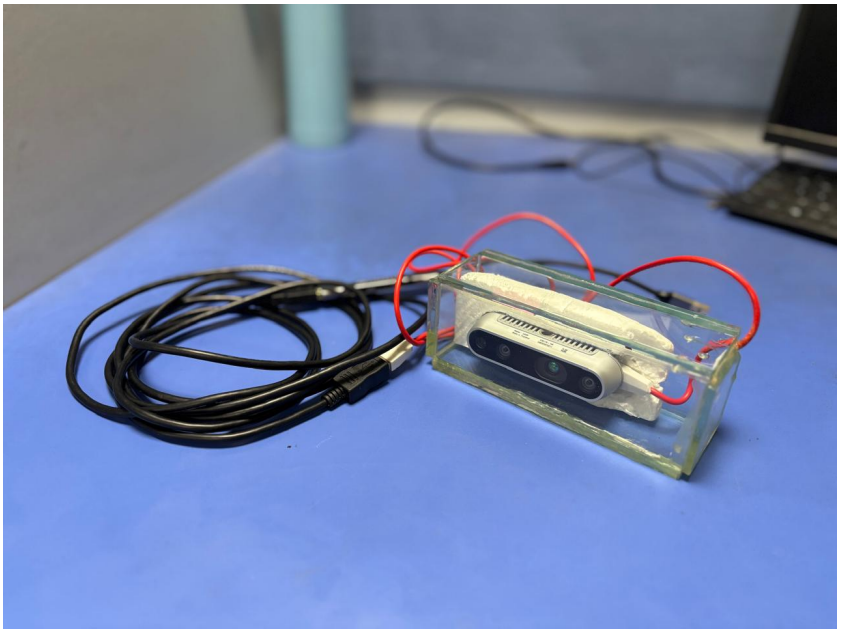

FIGURE 3.2: Underwater camera setup

structure comprises thick glass panes bonded with epoxy glue, drawing inspiration from aquarium designs. Thick, heavy glasses are used to counter the buoyancy caused by the air bubble inside the structure. The setup is connected with a 3 meter USB 2.0 cable for communication.

3.2 Data Acquisition

Two methods are implemented for data acquisition.

1. Underwater images captured using the camera setup.
2. Web scraping (exclusively for bot images for better generalization).

The environment for data acquisition is chosen as 10 10-foot by 8-foot by 4-foot inflatable family pool. The camera is submerged and moved by hand inside the pool. Artificial scenes are created inside the pool with the following classes of objects:

1. Rocks
2. Plastic bottle
3. Plastic bag
4. Metal
5. Bot

Images captured from various point of view is added to the dataset.

3.2.1 Annotation

For image annotation Roboflow platform is used. Roboflow is a platform for building and deploying computer vision models, especially for tasks like object detection, classification, and segmentation. It's widely used to simplify the data management, annotation, training, and deployment pipeline for machine learning models in computer vision.
Dataset contain a total of 5149 images across 5 classes.

Classes	**Annotations**
Rocks	2185
Plastic bottle	2175
Plastic bag	1400
Bot	1087
Metal	111

TABLE 3.1: No of annotations per classes

A split of 70-20-10 is used for train-validation-test split.

3.2.2 Augmentation

Images in the dataset are augmented to enhance the generalization and robustness of the models. The following augmentation methods are implemented on the dataset:

1. Rotation
2. Shearing
3. Hue variation
4. Brightness variation
5. Blur
6. Noise

The resultant dataset has 12359 images across all classes.

3.3 System Overview

The system is a complex video pipeline that integrates the YoloV11 object detection model, Inertial measurement unit, and depth map to give an accurate position of the target object with respect to the origin of the operation. The components of the pipeline are:

1. Intel Realsense d435i camera in a waterproof enclosure.
2. Fine-tuned and optimized yolo v11 model.
3. ByteTrack based object tracker
4. Depth estimation and filtering module
5. IMU fusion using Mahony filter
6. Visualization
7. Data logging

3.3.1 Data flow in pipeline

1. Sensor input
2. object detection
3. object tracking
4. depth estimation for bounding boxes
5. 3D coordinate computation
6. IMU fusion
7. World corrdinate transformation
8. Visualization
9. data logging

the entire pipeline is represented in figure 3.3.

3.4 Image pre-processing

Underwater vision presents a range of challenges due to the unique properties of water. Some of the noise is caused by the scattering of light, suspended particles, low visibility, and regularly changing surroundings. Some of this can be overcome using image processing techniques. The objective of the image processing module here is to improve the detection efficiency of the Yolo model. so the following image processing setup is adopted:

3.4.1 Gaussian Blur

It is a linear, low-pass, smoothing filter that filters out the image's high-frequency noise. In Gaussian blur, each pixel is replaced with a weighted average of its neighbors, with weights taken from a Gaussian distribution.

$$\mathrm{G(x,y)} = \frac{1}{2\pi\sigma^2}\mathrm{e}^{-\frac{\mathrm{x}^2+\mathrm{y}^2}{2\sigma^2}}$$

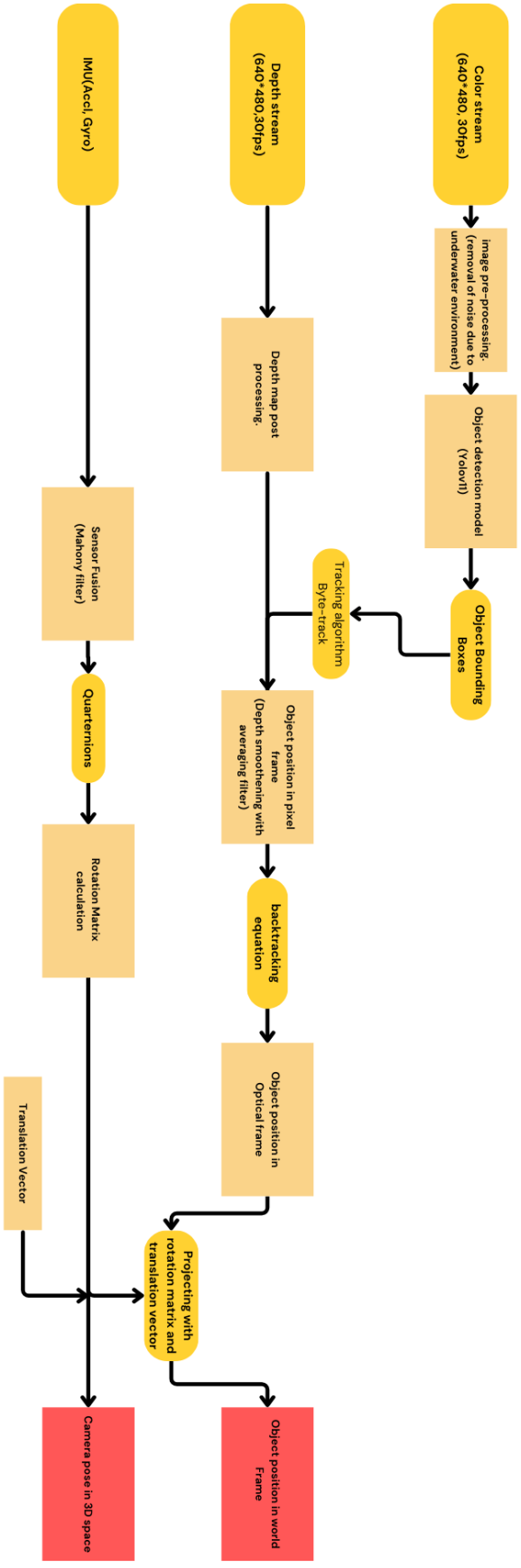

FIGURE 3.3: System Flowchart

Here sigma controls the degree of blurring. lower the value lower the blur.

3.4.2 Unsharp Masking technique

Applying gaussian blur reduces the edges in the image, but edges are required for efficient object detection with yolo model. So, an unsharp masking filter is used to enhance edges.

$$\text{Output Image} = \text{x} \cdot \text{Original} + \text{y} \cdot \text{Blurred} \tag{3.1}$$

here sum of x and y should be equal to one. i the project x is given the value 1.5 and y is given the value -0.5.

3.4.3 Bilateral Filter

Bilateral filer is an edge-preserving smoothing filter.it helps smooth textures like noise in water or lighting artifacts. It smooths out minor pixel level noise without undoing the edge clarity added by the unsharp masking.

3.4.4 Contrast Limited Adaptive Histogram Equalization (CLAHE)

It is used to improve the contrast of images. In traditional methods, the contrast of the whole image changes, but CLAHE works by dividing the image into smaller parts and adjusting the contrast in each part separately. This helps prevent the image from getting too bright or too dark in some areas.

After converting the image to lab color space, CLAHE is applied to the L(Lightness) channel. Underwater images suffer from poor contrast and color distortion; CLAHE improves local contrast and avoids amplifying noise. It is applied only to the l channel to avoid color distortion. This keeps color natural while enhancing visible details in bright and dark regions.

3.4.5

3.5 Model and Inference Setup

The object detection model chosen is Yolo V11. Yolo v11 is a state-of-the-art object detection model by Ultralytics. Yolo V11 offers fewer parameters than its predecessors while not compromising performance. A reduced number of parameters improves the inference speed. Ultralytics offers five different models: Yolo v11 nano, small, medium, large, and extra-large. All models are trained using the under water dataset. The hyperparameters and hardware specification are given below:
Device: Nvidia Geforce RTX 4050Ti- 6gb Gpu

1. Learning rate: 0.005
2. Epocs: 50
3. Image size: 640
4. Batch: 8(for medium model)
5. Optimizer : Adam

The extra-large model was trained on 2 x T4 gpus from kaggle.

3.5.1 Model Optimization

The finetuned models undergo quantization to optimize their performance and efficiency, specifically being converted to F16 (16-bit floating point) and int8 (8-bit integer) precision formats. This process reduces the model size and increases inference speed, making it more suitable for deployment, especially in environments with limited computational resources. The quantization is facilitated through the Ultralytics API, which provides a streamlined and user-friendly interface for implementing advanced techniques in model optimization. After quantization the models are validated using the underwater dataset.

3.6 Depth Estimation and Filtering

The Intel realsense camera offers onboard depth map estimation. It uses stereo vision to estimate the depth map. In stereo vision depth is calculated using triangulation:

$$\text{Depth} = \frac{\text{f} \cdot \text{B}}{\text{d}} \tag{3.2}$$

The camera computes disparity, and the above equation calculates the depth map. An attempt was made to compute the depth map directly from the left and right stereo images available from the camera. A neural network-based estimator, FadNet, was implemented. The result far surpassed the camera's onboard depth estimation. Ultimately, the onboard calculated depth map was chosen as the NN-based model requires higher computing resources while the real sense map requires none.

The object in the image is located using the bounding boxes provided by the Object detection model. The color image feed is aligned with the depth map, and the median depth in the region of interest is calculated. The depth estimated in this method was prone to significant noise, so a moving average filter with a window size of 10 was assigned to smooth the depth values.

3.7 Imu Based Orientation Estimation

The inertial measurement unit of Intel realsense camera contain following sensors:

1. 3-axis Gyroscope
2. 3-axis Accelerometer

To compute the rotation matrix from these sensor data sensor fusion is implemented. The following methods for sensor fusion are experimented:

3.7.1 Complementary filter

Complementary filter is the simplest method for sensor fusion. Here accelerometer gives a low-frequency data while gyroscope gives high frequency data. We combine this data together using a simple complementary equation:

$$\theta = \alpha \cdot \theta_{\text{gyro}} + (1 - \alpha) \cdot \theta_{\text{accel}} \tag{3.3}$$

3.7.2 Mahony Filter

The Mahony filter uses a feedback control mechanism to correct the estimated orientation using the error between the measured and estimated gravity/magnetic direction. The angular velocity is corrected by:

$$\boldsymbol{\omega}_{\text{corrected}} = \boldsymbol{\omega} + \mathrm{K_p} \cdot \mathrm{c} + \mathrm{K_i} \cdot \int \mathbf{e}\, \mathrm{dt} \tag{3.4}$$

- ω: gyroscope measurement
- **e** :orientation error (between measured gravity and estimated gravity)
- $\mathrm{K_p}$:proportional gain
- $\mathrm{K_i}$i : integral gain

Then the quarternion Q is udated using the corrected angular velocity.

3.7.3 Magwick Filter

The Madgwick filter uses gradient descent to minimize errors between measured and estimated gravity/magnetic field directions. It is designed to be computationally efficient and more accurate than Mahony in dynamic conditions.

$$\frac{\mathrm{d}\mathbf{q}}{\mathrm{dt}} = \frac{1}{2}\,\mathbf{q} \otimes \boldsymbol{\omega} - \beta \nabla \mathrm{f}(\mathbf{q}) \tag{3.5}$$

- ω: angular velocity

- $\mathbf{q}$: current Quarternion
- $\nabla f(\mathbf{q})$: gradient of cost function w.r.t. quaternion
- β: gain controlling convergence speed

Then integrate the gradient with a small timestep to get the quarternions.

3.7.4 Rotation matrix from Quaternions

we can compute rotation matrix from quarternions using the following equation:

$$R(q) = \begin{bmatrix} 1-2(y^2+z^2) & 2(xy-zw) & 2(xz+yw) \\ 2(xy+zw) & 1-2(x^2+z^2) & 2(yz-xw) \\ 2(xz-yw) & 2(yz+xw) & 1-2(x^2+y^2) \end{bmatrix}$$

the resultant matrix is used to transform the point from camera frame to world frame.

3.8 Position Estimation

Now, we have pixel frame coordinates and the distance from the camera to the target (depth). However, this does not provide the real location of the object. The coordinates need to be transformed into world frame, but before that, the pixel frame needs to be converted into a camera frame. That can be done using back-projection. Back projection is a method to perform this conversion using the camera intrinsics and the depth information. The equation is as follows:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = Z \cdot K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

where,

- K is camera intrinsics matrix (f_x and f_y are focal lengths and C_x and c_y are optical centers)
- (X,Y,Z): 3D point in camera frame
- (u,v): pixel coordinates
- Z: depth at pixel coordinated

3.8.1 World coordinate from rotation matrix

We have the coordinates in the camera coordinates now. We need to convert it into world coordinates. We can do that using the rotation matrix. However, we need to account for the translation of the source. We introduce a new vector, T, which is a translation vector with the coordinated for the source in the world frame. For this calculation, there is an assumption: The source is parallel to the world axes at the beginning of the operation. With the following equation, we can calculate the world coordinates of the target object.

$$P_w = R \cdot P_c + T \tag{3.6}$$

where,

- p_w is the point in world coordinate
- R is the rotation matrix
- T is the translation matrix
- p_c is point in camera coordinates

3.9 Data Logging

World coordinate of the target object is saved into a CSV (Comma-Separated Values) file with corresponding timestamp and tracking id for each of the frames. This data can be utilized to calculate the trajectory of the target object.

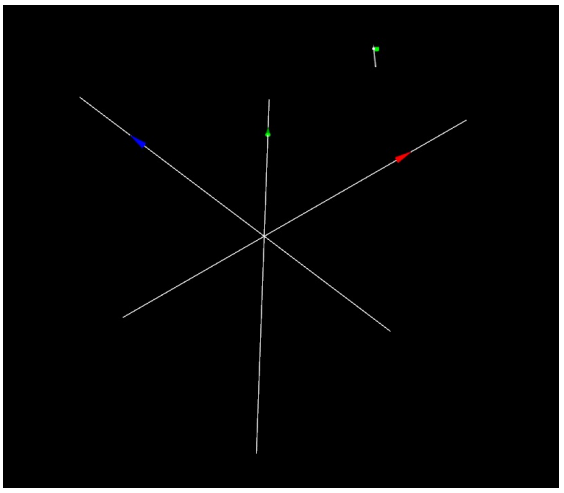

FIGURE 3.4: 3D representation of object location

3.10 3D visualization

To provide real-time visual feedback of object localization and orientation in the 3d space, the systemm integrates a 3D visualization modeule using Open3D. It generates a 3 dimensional space and plots the realtime location of the target object and camera. It also shows the oriwntation of the system in realtime. it has a memory of upto 10 data points. a representation is shown in the figure 3.4.

1. X-axis : Red
2. y-axis : green
3. z-axis : blue
4. green dot: target position

Chapter 4
Results and Discussion

The objective of the project is to achieve maximum possible latency for the video processing pipeline while not compromising on the accuracy. Various metrics are used in evaluation of the different modules of the pipeline.

All the relevant code, dataset, trained models and test samples are uploaded in the following Links Dataset, Google Drive.
The official git repository for the project: Github

4.1 Underwater Dataset

As mentioned earlier the underwater dataset contains 12359 images across 5 classes. A few images and their annotations are given in figure 4.1:

- Bot ID: 0
- metal ID: 1
- plastic bags ID: 2
- plastic bottle ID: 3

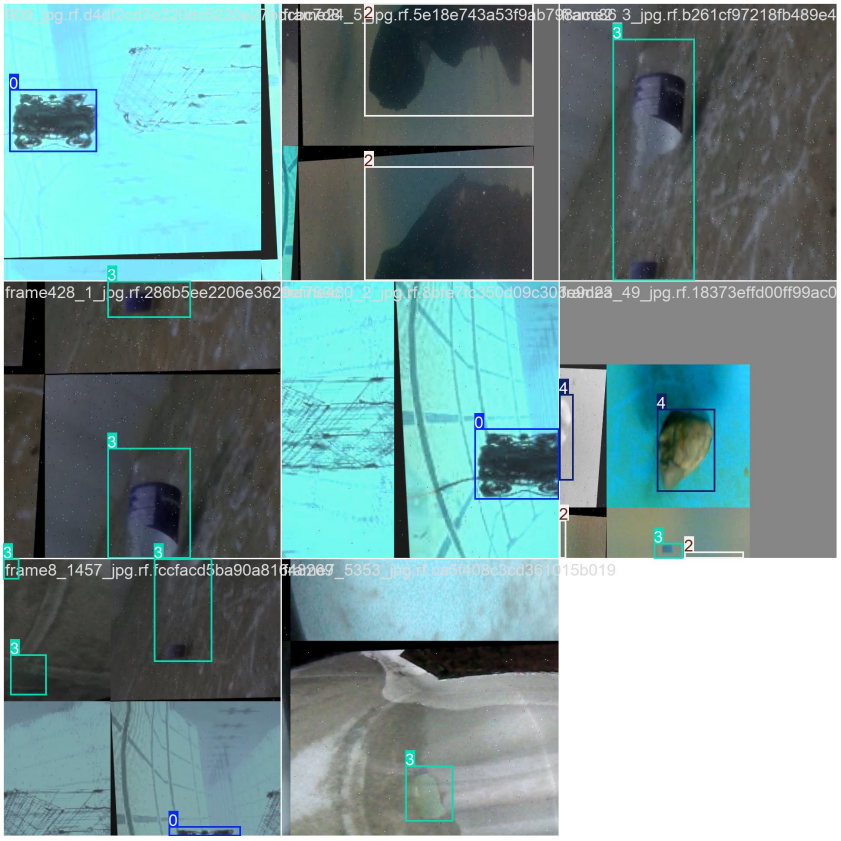

FIGURE 4.1: Dataset

- rock ID: 4

4.2 Image pre processing

The image processing pipeline acts as a pre-processing technique prior to the model inference. The filters involved are listed in the previous section. some images before and after the noise removal and enhancement is shown in figure 4.2.

4.3 Model Training

All 5 available architectures of yolo v11 are fine tuned on the underwater dataset. The hyperparameters for training are listed in the previous section. The training results are as follows:

FIGURE 4.2: image pre-processing left: unprocessed right: processed

Model	**Precision**	**Recall**	**mAP 0.5**
Yolo11n	0.6214	0.7371	0.8632
Yolo11s	0.6685	0.7704	0.8862
Yolo11m	0.6214	0.7809	0.9149
Yolo11l	0.6456	0.7612	0.8457
Yolo11x	0.6559	0.7706	0.8862

TABLE 4.1: Evaluation matrices for model

The higher results in evaluation is due to the fact that the experiment is local to one environment and the dataset is generated in the same environment. Therefore such results are expected.

4.4 Model Optimization

Quantization refers to the process of reducing the precision of the numbers used to represent model weights and activations. By converting floating-point numbers (such as float32) to lower precision formats like float16 (f16) and int8, we can significantly decrease the model size and improve inference speed without sacrificing much accuracy. Fine-tuned models are quantized with the help of tensor and onyx libraries. The models

are exported to tensorrt engine format with f16 and int8 precisions. The exported models are then validated using the underwater dataset.

4.5 Evaluation of Pipeline

The video processing pipeline should be evaluated by how fast each frames are processed. This can be done by calculating the latency. The time for execution is calculated by the help of time library. A .bag file containing sensor data is recorded and the same file is used to process all the model variants. This ensures unbiased results. The latency for each frame processed is calculated and the mean is found out. The inverse of average latency gives us the total number of frames that can be processed. Since the camera output is capped at 30 frame per second (USB 2.0 limits the stream options) we cannot find out the total program latency. During inspection of the output feed its visible that even though the loss in performance of the models are negligible the number for frames which object is detected is fewer. So a newer parameter is introduced which is average detections. This is assuming that there are no misclassifications of the objects and the target is present in most of the frames. This parameter may provide a better insight into the performance of the pipeline.

All the different configurations and its performance given in the table 4.2. From the table we can easily find that Yolov11m model with int8 quantization gives the best tradeoff between accuracy and latency.

4.6 Visualization

The calculated 3d coordinate of the object and source position is plotted using open3D library. A representation of the same is shown in the figure.

Model	Quantization	precision	recall	mAp	latency (ms)	FPS	detection %
Yolo11n	mixed	0.6214	0.7371	0.8632	53	18.58	0.62
Yolo11s	mixed	0.6685	0.7704	0.8862	62	15.90	0.81
Yolo11m	mixed	0.6214	0.7809	0.9149	72	13.85	0.86
Yolo11l	mixed	0.6456	0.7612	0.8457	80	12.35	0.83
Yolo11x	mixed	0.6559	0.7706	0.8862	76	13.154	0.79
Yolo11n	F16	0.6545	0.7789	0.8818	38	26	0.56
Yolo11s	F16	0.6688	0.7700	0.9034	41	24.26	0.71
Yolo11m	F16	0.6685	0.7810	0.9171	39	25.03	0.82
Yolo11l	F16	0.6787	0.7658	0.8917	39	25.13	0.81
Yolo11x	F16	0.6563	0.7698	0.8862	39.2	25.47	0.7457
Yolo11n	Int8	0.6348	0.7675	0.8709	22	44.63	0.56
Yolo11s	Int8	0.5822	0.6708	0.7705	15	64.34	0.014
Yolo11m	Int8	0.6588	0.7743	0.8860	33	29.82	0.78
Yolo11l	Int8	0.6475	0.7643	0.8652	36	27.05	0.76
Yolo11x	Int8	0.6260	0.7586	0.8432	38	26	0.786

TABLE 4.2: Model Validation Data

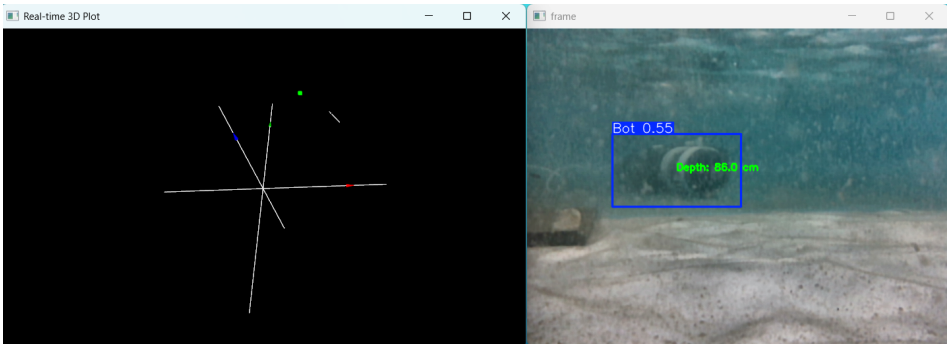

FIGURE 4.3: 3D point cloud and image.

Chapter 5
Conclusion

The project successfully demonstrates a real-time, resource efficient pipeline for underwater bot detection and localization. It integrates visual and inertial sensor data to perform accurate 3 dimensional localization. The system leverages YoloV11 object detection model optimized via tensorrt quantization to increase inference speed along with inertial measurement unit fusion and depth sensing for precise spatial awareness. The pipeline includes various image processing techniques to improve detection such as CLAHE, bilateral filter and unsharp mask to effectively overcome the challenges underwater.

A comprehensive dataset was captured, annotated and augmented to train the models. Among them quantized YoloV11 medium model is identified as the optimal tradeoff between accuracy and inference speed, achieving an mAP of 88.6% and nearly 30 frame per second.

The results demonstrate that the developed system is robust, scalable and well-suited for real-time deployment in resource-constrained autonomous underwater vehicles. This work not only contributes a practical implementation pipeline for underwater perception but also lays the groundwork for future extensions in underwater mapping, autonomous navigation, and object interaction tasks.

Bibliography

[1] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*, vol. 1, 2001, pp. I–I.

[2] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *2014 IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 580–587.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," 2016. [Online]. Available: https://arxiv.org/abs/1506.02640

[4] G. Jocher, J. Qiu, and A. Chaurasia, "Ultralytics yolo," 2023, aGPL-3.0 License. [Online]. Available: https://ultralytics.com

[5] B. Dwyer, J. Nelson, T. Hansen *et al.*, "Roboflow (version 1.0)," 2024, computer vision. [Online]. Available: https://roboflow.com

[6] H. Wang and N. Xiao, "Underwater object detection method based on improved faster rcnn," *Applied Sciences*, vol. 13, p. 2746, 02 2023.

[7] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, p. 1680–1716, Nov. 2023. [Online]. Available: http://dx.doi.org/10.3390/make5040083

[8] S. Srivastava, A. V. Divekar, C. Anilkumar *et al.*, "Comparative analysis of deep learning image detection algorithms," *Journal of Big Data*, vol. 8, p. 66, 2021. [Online]. Available: https://doi.org/10.1186/s40537-021-00434-w

[9] L. Wang, X. Ye, S. Wang, and P. Li, "Ulo: An underwater light-weight object detector for edge computing," *Machines*, vol. 10, no. 8, 2022. [Online]. Available: https://www.mdpi.com/2075-1702/10/8/629

[10] A. Balaji, Y. S, K. CK, N. R, G. Dooly, and S. Dhanalakshmi, "Deep wavenet-based yolo v5 for underwater object detection," in *OCEANS 2023 - Limerick*, 2023, pp. 1–5.

[11] Y. Fan, L. Zhang, and P. Li, "A lightweight model of underwater object detection based on yolov8n for an edge computing platform," *Journal of Marine Science and Engineering*, vol. 12, no. 5, 2024. [Online]. Available: https://www.mdpi.com/2077-1312/12/5/697